

Implementierungsleitfaden: Von JSON-Dropfolder zu Graph-RAG (BTC/ETH/SOL)

1. Zielbild und Überblick

Ziel ist eine durchgängige Pipeline, die folgende Schritte automatisiert:

1. **Manuelle Erstellung** von Episoden-JSONs (Segmentierer = `base_*.json`, Spezialist = `refined_*.json`).
2. **Ablage im Dropfolder** auf dem Unraid-Server.
3. **Automatische Verarbeitung in n8n**:
4. Erkennen von `base_` vs. `refined_` Dateien.
5. Schema-Validierung und Archivierung.
6. Schreiben der Episoden in PostgreSQL (`crypto_market_events` + optionale Metriktabelle).
7. Berechnung quantitativer Kennzahlen pro Event aus der Kursdaten-DB.
8. Ex-Post-Updates von Strategie-Feldern (optional, später).
9. Projektion der Events nach Neo4j (Graphmodell).
10. Erzeugen von Embeddings mit Ollama und Speichern in Qdrant.
11. **Graph-RAG-Service** (separater kleiner Dienst), der:
12. eine Query entgegennimmt,
13. mit Ollama embeddings erzeugt,
14. über Qdrant ähnliche Events findet,
15. zusätzliche Struktur aus Neo4j lädt,
16. optional aktuelle Marktdaten aus PostgreSQL bezieht,
17. und daraus mit einem LLM eine Antwort generiert.

Komponenten auf Ihrem Unraid-Server:

- n8n (Orchestrierung / ETL)
- PostgreSQL 17 (Kursdaten + Events + Metriken)
- Neo4j (Graphmodell)
- Qdrant (Vektorindex)
- Ollama (lokales Embedding-Modell, optional auch LLM-Modell)

2. Verzeichnis- und Dateistruktur

2.1 Dropfolder und Archive

Empfohlene Ordnerstruktur auf dem Unraid-Server (als Shares/Volumes, in n8n gemountet):

- `/data/llm_events/drop/` → Eingehende JSON-Dateien (manuell abgelegt)
- `/data/llm_events/archive/base/` → Archivierte Segmentierer-Outputs
- `/data/llm_events/archive/refined/` → Archivierte Spezialisten-Outputs
- `/data/llm_events/error/` → Dateien mit Validierungsfehlern

2.2 Namensschema der Dateien

Konvention:

- `base_<asset>_<YYYY>_<MM><half>.json`
- `refined_<asset>_<YYYY>_<MM><half>.json`

Beispiele:

- `base_btc_2025_01h1.json` → BTC, 1. Januarhälfte 2025, Segmentierer
- `refined_eth_2025_09h2.json` → ETH, 2. Septemberhälfte 2025, Spezialist

RegEx für n8n (Filename-Parsing):

```
^(base|refined)_(btc|eth|sol)_(\d{4})_(\d{2})(h[12])\.json$
```

Aus dem Dateinamen extrahierbar:

- `kind` → `base` oder `refined`
- `asset` → `btc` / `eth` / `sol` (später in Großschrift `BTC` etc.)
- `year` → `YYYY`
- `month` → `MM`
- `half` → `h1` oder `h2`

3. Relationales Datenmodell (PostgreSQL)

3.1 Kursdaten (Beispielschema)

Ziel: Zeitreihen für OHLCV (z. B. 1-Stunden-Auflösung), die für Kennzahlen pro Event abgefragt werden können.

Beispieltabelle `ohlcv_1h`:

```
CREATE TABLE ohlcv_1h (  
  asset      text      NOT NULL,          -- 'BTC', 'ETH', 'SOL'  
  ts         timestamp NOT NULL,          -- Candle-Timestamp (z. B.  
Stunde)  
  open       numeric   NOT NULL,  
  high       numeric   NOT NULL,  
  low        numeric   NOT NULL,  
  close      numeric   NOT NULL,  
  volume     numeric   NOT NULL,  
  PRIMARY KEY (asset, ts)  
);
```

Weitere Tabellen (optional):

- funding_1h(asset, ts, rate)
- oi_1h(asset, ts, open_interest)
- onchain_metrics(asset, ts, metric_name, metric_value)

3.2 Episoden-Tabelle crypto_market_events

Diese Tabelle speichert eine Zeile pro Episode (Event) mit allen LLM-Feldern.

Beispiel-DDL:

```
CREATE TABLE crypto_market_events (  
  event_id          text PRIMARY KEY,  
  asset             text NOT NULL,  
  period_start      date NOT NULL,  
  period_end        date NOT NULL,  
  time_start        timestampz NOT NULL,  
  time_end          timestampz NOT NULL,  
  
  market_regime     text,  
  market_phase      text,  
  volatility_context text,  
  leverage_level     text,  
  liquidity_condition text,  
  sentiment_state    text,  
  
  event_name         text,  
  event_category     text,  
  event_subcategory  text,  
  
  surprise_factor    integer,  
  invalidation        boolean,  
  primary_mechanism  text,  
  mechanism_detail   text,  
  mechanism_strength integer,  
  cross_asset_corr   text,  
  signal_type_overall text,  
  leading_indicators text,  
  
  pro_strategy       text,  
  strategy_outcome_rating integer,  
  strategy_timing    text,  
  risk_profile       text,  
  impact_strength    integer,  
  confidence_score    integer,  
  similar_pattern_refs text,  
  
  data_origin        text,  
  strategy_source     text,
```

```

        created_at          timestampz,
        updated_at          timestampz
    );

```

Hinweis: `leading_indicators` und `similar_pattern_refs` können später auf `jsonb` umgestellt werden.

3.3 Quantitative Metriken pro Event (optional, empfohlen)

Separat, um die Episode-Tabelle „sauber“ zu halten.

```

CREATE TABLE event_metrics (
    event_id          text PRIMARY KEY
    REFERENCES crypto_market_events(event_id),
    event_return_pct   numeric,    -- (price_end / price_start - 1)
    event_max_drawdown_pct numeric,
    event_max_runup_pct numeric,
    event_realized_vol numeric,

    created_at        timestampz DEFAULT now(),
    updated_at        timestampz DEFAULT now()
);

```

3.4 Optionale Rohfile-Tabelle

Zur Nachvollziehbarkeit der verarbeiteten Files:

```

CREATE TABLE raw_files (
    filename          text PRIMARY KEY,
    kind              text,        -- 'base' oder 'refined'
    asset             text,
    period_start      date,
    period_end        date,
    half              text,        -- 'h1' oder 'h2'
    inserted_at       timestampz DEFAULT now(),
    notes             text
);

```

4. Graphmodell in Neo4j (Konzept)

4.1 Knoten-Typen (Labels)

Empfohlenes Grundschema:

- `(:Asset {symbol})` → z. B. `BTC`, `ETH`, `SOL`

- (:Event {event_id, event_name, ...}) → eine Episode
- (:Regime {name}) → strong_bull, bull, neutral, bear, strong_bear
- (:Phase {name}) → accumulation, markup, distribution, markdown, ranging
- (:Mechanism {name}) → z.B. Short Squeeze, Spot Absorption, ...
- optional weitere Knoten: (:Category), (:SubCategory), (:Indicator)

4.2 Relationen

Typische Kanten:

- (e:Event)-[:FOR_ASSET]->(a:Asset)
- (e)-[:HAS_REGIME]->(r:Regime)
- (e)-[:HAS_PHASE]->(p:Phase)
- (e)-[:HAS_MECHANISM]->(m:Mechanism)
- (e1)-[:SIMILAR_TO {source, score}]->(e2)

Später können zusätzlich temporale Kanten (:PRECEDES, :FOLLOWS) ergänzt werden.

5. Qdrant: Vektorindex für Episoden

5.1 Collection-Design

Collection: crypto_events

- Vektor-Dimension: abhängig vom verwendeten Ollama-Embedding-Modell (z. B. 768).
- Distanzmetrik: z. B. Cosine.
- Payload-Felder pro Event-Vektor:
 - event_id
 - asset
 - time_start
 - time_end
 - event_name
 - market_regime
 - market_phase

5.2 Textrepräsentation für Embedding

Pro Event wird ein zusammenfassender Text gebaut, z. B.:

"Event: The Six-Figure Breakout. Asset: BTC. Regime: strong bull, phase: markup. Mechanism: short squeeze and spot-driven breakout. Strategy: momentum long with aggressive profit taking."

Dieser Text wird an Ollama geschickt, um einen Vektor zu erzeugen.

6. n8n-Workflow: Vom Dropfolder nach PostgreSQL, Neo4j und Qdrant

6.1 Trigger-Node

Verwenden Sie entweder:

- **Filesystem-Trigger** (z. B. Community-Node) mit „on file added“ im Pfad `/data/llm_events/drop/`, oder
- einen **Cron-Trigger**, der regelmäßig „List Files“ im Drop-Ordner ausführt.

Der Trigger soll pro gefundener Datei genau ein n8n-Item erzeugen mit:

- `binary.data` → Dateiinhalt
- `json.filename` → Dateiname (z. B. `refined_btc_2025_01h1.json`)

6.2 Node 1: Function – Dateiname parsen

Zweck: Art der Datei (base/refined) und Metadaten extrahieren.

Beispiel-Code (JavaScript in n8n-Function-Node):

```
const item = items[0];
const filename = item.json.filename;

const regex = /^(base|refined)_(btc|eth|sol)_(\d{4})_(\d{2})(h[12])\.json$/;
const m = filename.match(regex);

if (!m) {
    throw new Error(`Ungültiger Dateiname: ${filename}`);
}

const [, kind, assetLower, year, month, half] = m;

item.json.kind = kind; // 'base' oder 'refined'
item.json.asset = assetLower.toUpperCase(); // 'BTC', 'ETH', 'SOL'
item.json.year = year;
item.json.month = month;
item.json.half = half; // 'h1' oder 'h2'

return items;
```

6.3 Node 2: IF oder Switch – Routing base vs. refined

- IF: `json.kind == 'base'` → Pfad A
- ELSE → Pfad B (`refined`)

6.4 Pfad A – base_*.json (Segmentierer-Output)

6.4.1 JSON parsen

Node: **Move Binary Data**

- Mode: Binary → JSON
- Source Key: `data`

Ergebnis: `items[0].json` enthält direkt das JSON-Objekt mit `asset`, `period_start`, `period_end`, `episodes`.

6.4.2 Validierung (leicht)

Node: **Function**

Beispielhafte Minimalvalidierung:

```
const item = items[0];
const data = item.json;

if (!data.asset || !data.period_start || !data.period_end || !
Array.isArray(data.episodes)) {
  throw new Error('Missing top-level fields in base JSON');
}

const allowedRegimes = ['strong_bull', 'bull', 'neutral', 'bear',
'strong_bear'];
const allowedPhases = ['accumulation', 'markup', 'distribution', 'markdown',
'ranging'];
const allowedRisk = ['high_risk', 'moderate', 'conservative', 'hedge_only'];

const warnings = [];

for (const ep of data.episodes) {
  if (!ep.event_id) warnings.push('event_id missing');
  if (!allowedRegimes.includes(ep.market_regime)) {
    warnings.push(`Invalid market_regime for ${ep.event_id}: $
{ep.market_regime}`);
  }
  if (!allowedPhases.includes(ep.market_phase)) {
    warnings.push(`Invalid market_phase for ${ep.event_id}: $
{ep.market_phase}`);
  }
  if (!allowedRisk.includes(ep.risk_profile)) {
    warnings.push(`Invalid risk_profile for ${ep.event_id}: $
{ep.risk_profile}`);
    ep.risk_profile = 'moderate';
  }
}
```

```

    ep.surprise_factor      = Math.max(0, Math.min(100,
ep.surprise_factor      ?? 0));
    ep.mechanism_strength   = Math.max(0, Math.min(100,
ep.mechanism_strength   ?? 0));
    ep.impact_strength     = Math.max(0, Math.min(100,
ep.impact_strength     ?? 0));
    ep.strategy_outcome_rating = Math.max(1, Math.min(5,
ep.strategy_outcome_rating ?? 3));
    ep.confidence_score     = Math.max(1, Math.min(5,
ep.confidence_score     ?? 3));
}

data.validation_warnings = warnings;

return items;

```

Optional: Bei harten Fehlern in `/data/llm_events/error/` verschieben.

6.4.3 Archivierung

Node: **Write Binary File** oder **Move File**

- Quelle: ursprüngliche Binary
- Ziel: `/data/llm_events/archive/base/<filename>`

Optional: Eintrag in `raw_files` (Postgres-Node → `INSERT INTO raw_files (...) VALUES (...)`).

Hinweis: Die eigentliche weitere Verarbeitung ist an die `refined_*.json` gekoppelt. Sie können hier bereits Inserts in `crypto_market_events` machen, müssen es aber nicht.

6.5 Pfad B – refined_*.json (Spezialisten-Output)

6.5.1 JSON parsen

Auch hier: **Move Binary Data**, Binary → JSON.

6.5.2 Matching zur base-Datei (optional, aber empfohlen)

Node: **Function**

```

const item = items[0];
const { asset, year, month, half } = item.json; // aus dem ersten Function-
Node

const baseFileName = `base_${asset.toLowerCase()}_${year}_${month}${
half}.json`;
item.json.baseFileName = baseFileName;
item.json.baseFilePath = `/data/llm_events/archive/base/${baseFileName}`;

```



```
return items;
```

Anschließend optional:

- Node: **Read Binary File** mit `filePath = {{$json["baseFilePath"]}}`
- Bei Fehler → IF-Node → Fehlerpfad (z. B. in `/error/` verschieben, Log).
- Bei Erfolg → Sie haben sowohl refined-JSON als auch die archivierte base-Version im Flow.

6.5.3 Episodes in Postgres schreiben

1. Node: **Function** – Episodes „aufsplitten“

```
const data = items[0].json; // refined JSON
const output = [];

for (const ep of data.episodes) {
  const row = {
    event_id: ep.event_id,
    asset: data.asset,
    period_start: data.period_start,
    period_end: data.period_end,
    time_start: ep.time_start,
    time_end: ep.time_end,
    market_regime: ep.market_regime,
    market_phase: ep.market_phase,
    volatility_context: ep.volatility_context,
    leverage_level: ep.leverage_level,
    liquidity_condition: ep.liquidity_condition,
    sentiment_state: ep.sentiment_state,
    event_name: ep.event_name,
    event_category: ep.event_category,
    event_subcategory: ep.event_subcategory,
    surprise_factor: ep.surprise_factor,
    invalidation: ep.invalidation,
    primary_mechanism: ep.primary_mechanism,
    mechanism_detail: ep.mechanism_detail,
    mechanism_strength: ep.mechanism_strength,
    cross_asset_corr: ep.cross_asset_corr,
    signal_type_overall: ep.signal_type_overall,
    leading_indicators: ep.leading_indicators,
    pro_strategy: ep.pro_strategy,
    strategy_outcome_rating: ep.strategy_outcome_rating,
    strategy_timing: ep.strategy_timing,
    risk_profile: ep.risk_profile,
    impact_strength: ep.impact_strength,
    confidence_score: ep.confidence_score,
    similar_pattern_refs: ep.similar_pattern_refs,
    data_origin: 'web_search_llm',
    strategy_source: 'specialist_refined',
  };
  output.push(row);
}
```

```

        created_at: ep.created_at,
        updated_at: ep.updated_at,
    };
    output.push({ json: row });
}

```

```
return output;
```

1. Node: **Postgres** – INSERT ... ON CONFLICT

Query-Beispiel:

```

INSERT INTO crypto_market_events (
    event_id, asset, period_start, period_end, time_start, time_end,
    market_regime, market_phase, volatility_context, leverage_level,
    liquidity_condition, sentiment_state,
    event_name, event_category, event_subcategory,
    surprise_factor, invalidation, primary_mechanism, mechanism_detail,
    mechanism_strength,
    cross_asset_corr, signal_type_overall, leading_indicators,
    pro_strategy, strategy_outcome_rating, strategy_timing, risk_profile,
    impact_strength, confidence_score,
    similar_pattern_refs, data_origin, strategy_source, created_at, updated_at
) VALUES (
    {{ $json.event_id }}, {{ $json.asset }}, {{ $json.period_start }}, {{
$json.period_end }},
    {{ $json.time_start }}, {{ $json.time_end }},
    {{ $json.market_regime }}, {{ $json.market_phase }}, {{
$json.volatility_context }},
    {{ $json.leverage_level }}, {{ $json.liquidity_condition }}, {{
$json.sentiment_state }},
    {{ $json.event_name }}, {{ $json.event_category }}, {{
$json.event_subcategory }},
    {{ $json.surprise_factor }}, {{ $json.invalidation }}, {{
$json.primary_mechanism }},
    {{ $json.mechanism_detail }}, {{ $json.mechanism_strength }},
    {{ $json.cross_asset_corr }}, {{ $json.signal_type_overall }}, {{
$json.leading_indicators }},
    {{ $json.pro_strategy }}, {{ $json.strategy_outcome_rating }}, {{
$json.strategy_timing }},
    {{ $json.risk_profile }}, {{ $json.impact_strength }}, {{
$json.confidence_score }},
    {{ $json.similar_pattern_refs }}, {{ $json.data_origin }}, {{
$json.strategy_source }},
    {{ $json.created_at }}, {{ $json.updated_at }}
)
ON CONFLICT (event_id) DO UPDATE SET
    pro_strategy = EXCLUDED.pro_strategy,
    strategy_outcome_rating = EXCLUDED.strategy_outcome_rating,
    strategy_timing = EXCLUDED.strategy_timing,

```

```

risk_profile = EXCLUDED.risk_profile,
impact_strength = EXCLUDED.impact_strength,
confidence_score = EXCLUDED.confidence_score,
similar_pattern_refs = EXCLUDED.similar_pattern_refs,
strategy_source = EXCLUDED.strategy_source,
updated_at = EXCLUDED.updated_at;

```

6.5.4 Quantitative Kennzahlen pro Event aus Kursdaten berechnen

Dieser Schritt kann in n8n direkt nach dem Insert erfolgen oder in einem separaten, zeitgesteuerten Workflow.

1. Node: **Postgres – Select aus ohlcv_1h**

Beispiel (vereinfacht, pro Event):

```

WITH prices AS (
  SELECT
    ts,
    close,
    low,
    high,
    FIRST_VALUE(close) OVER w AS price_start,
    LAST_VALUE(close) OVER w AS price_end
  FROM ohlcv_1h
  WHERE asset = $1
    AND ts BETWEEN $2 AND $3
  WINDOW w AS (ORDER BY ts
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
)
SELECT
  MIN(ts) AS ts_first,
  MAX(ts) AS ts_last,
  MIN(low) AS min_price,
  MAX(high) AS max_price,
  AVG(close) AS avg_price,
  MAX(price_end / price_start - 1) AS event_return_pct
FROM prices;

```

1. Node: **Postgres – Insert/Update event_metrics**

```

INSERT INTO event_metrics (
  event_id, event_return_pct
) VALUES (
  {{ $json.event_id }}, {{ $json.event_return_pct }}
)
ON CONFLICT (event_id) DO UPDATE SET
  event_return_pct = EXCLUDED.event_return_pct,
  updated_at = now();

```

Sie können später weitere Kennzahlen ergänzen (Drawdown, Volatilität etc.).

6.5.5 Neo4j-Upsert

Node: **HTTP Request** (Neo4j HTTP API)

- Methode: POST
- URL: z. B. `http://neo4j:7474/db/neo4j/tx/commit`
- Auth: Basic (Neo4j User/Pass)
- Body (JSON):

```
{
  "statements": [
    {
      "statement": "
MERGE (a:Asset {symbol: $asset})
MERGE (e:Event {event_id: $event_id})
SET e.event_name = $event_name,
    e.asset = $asset,
    e.time_start = datetime($time_start),
    e.time_end = datetime($time_end),
    e.market_regime = $market_regime,
    e.market_phase = $market_phase,
    e.impact_strength = $impact_strength,
    e.strategy_outcome_rating = $strategy_outcome_rating,
    e.confidence_score = $confidence_score
MERGE (e)-[:FOR_ASSET]->(a)
MERGE (r:Regime {name: $market_regime})
MERGE (e)-[:HAS_REGIME]->(r)
MERGE (p:Phase {name: $market_phase})
MERGE (e)-[:HAS_PHASE]->(p)
",
      "parameters": {
        "asset": "{{ $json.asset }}",
        "event_id": "{{ $json.event_id }}",
        "event_name": "{{ $json.event_name }}",
        "time_start": "{{ $json.time_start }}",
        "time_end": "{{ $json.time_end }}",
        "market_regime": "{{ $json.market_regime }}",
        "market_phase": "{{ $json.market_phase }}",
        "impact_strength": {{ $json.impact_strength }},
        "strategy_outcome_rating": {{ $json.strategy_outcome_rating }},
        "confidence_score": {{ $json.confidence_score }}
      }
    }
  ]
}
```

Analog können Sie später `:SIMILAR_TO` Kanten aus `similar_pattern_refs` erzeugen.

6.5.6 Embedding (Ollama) und Upsert nach Qdrant

1. Node: **Function** – Textrepräsentation bauen

```
const item = items[0];
const ep = item.json;

const text = `Event: ${ep.event_name}. Asset: ${ep.asset}. ` +
  `Regime: ${ep.market_regime}, phase: ${ep.market_phase}. ` +
  `Mechanism: ${ep.primary_mechanism}. ` +
  `Strategy: ${ep.pro_strategy}.`;

item.json.embedding_text = text;
return items;
```

1. Node: **HTTP Request** – Ollama Embedding

2. Methode: POST

3. URL: z. B. `http://ollama:11434/api/embeddings`

4. Body:

```
{
  "model": "<embedding-model-name>",
  "prompt": "{{ $json.embedding_text }}"
}
```

• Response: Vektor (Liste von Floats) → in `json.vector` speichern.

• Node: **HTTP Request** – Qdrant Upsert

• Methode: PUT/POST

• URL: z. B. `http://qdrant:6333/collections/crypto_events/points?wait=true`

• Body:

```
{
  "points": [
    {
      "id": "{{ $json.event_id }}",
      "vector": {{ $json.vector }},
      "payload": {
        "event_id": "{{ $json.event_id }}",
        "asset": "{{ $json.asset }}",
        "event_name": "{{ $json.event_name }}",
        "market_regime": "{{ $json.market_regime }}",
        "market_phase": "{{ $json.market_phase }}",
        "time_start": "{{ $json.time_start }}",
        "time_end": "{{ $json.time_end }}"
      }
    }
  ]
}
```

```
}  
}  
]  
}
```

6.5.7 Archivierung refined-File

Zum Abschluss:

- Node: **Write/Move File** → `/data/llm_events/archive/refined/<filename>`

Optional: Eintrag in `raw_files`.

7. Graph-RAG-Service (Query-Pfad)

Der Graph-RAG-Service wird als eigener kleiner Dienst (z. B. Python FastAPI) auf dem Unraid-Server betrieben.

7.1 Architekturübersicht

1. **HTTP-Endpoint**, z. B. `POST /graph_rag/query`
2. Request-Body:

```
{  
  "query": "Wie verhielt sich BTC in Phasen mit starken Short Squeezes?",  
  "asset": "BTC"  
}
```

1. Schritt 1: Embedding der Query mit Ollama

2. HTTP-Call an Ollama (`/api/embeddings`) mit Query-Text.

3. Schritt 2: Qdrant-Similarity-Search

4. Query-Vektor gegen `crypto_events` (Filter `asset == BTC`, `top_k = 10`).

5. Ergebnis: Liste von `event_id`s + Scores.

6. Schritt 3: Kontext aus Neo4j

7. Cypher-Query, um Events + Nachbarschaft (Regime, Phase, Mechanismen, ähnliche Events) zu holen.

8. Schritt 4: Optional weitere Daten aus PostgreSQL

9. Zusätzliche Kennzahlen aus `event_metrics` / Kursdatenbank (z. B. durchschnittliche Rückläufe, Drawdowns etc.).

10. Schritt 5: Prompt-Konstruktion + LLM-Call

11. Query + Top-Events + Graph-Kontext + Kennzahlen → zusammengefasster Prompt.

12. Aufruf eines LLM (Ollama oder externer Provider).

13. Schritt 6: Antwort an Client

14. Strukturierte Antwort mit:

15. verbaler Zusammenfassung,

16. referenzierten Events,

17. ggf. quantitativen Kennzahlen.

7.2 Beispiel-Pseudocode (Python-ähnlich)

```
def graph_rag(query: str, asset: str):  
    # 1) Query-Embedding  
    q_vec = embed_with_ollama(query)  
  
    # 2) Qdrant-Suche  
    top_events = qdrant_search(  
        collection="crypto_events",  
        vector=q_vec,  
        filter={"must": [{"key": "asset", "match": {"value": asset}}]},  
        top_k=10,  
    )  
  
    event_ids = [p["payload"]["event_id"] for p in top_events]  
  
    # 3) Neo4j-Kontext laden  
    graph_ctx = load_graph_context_from_neo4j(event_ids)  
  
    # 4) Event-Metriken aus Postgres  
    metrics = load_event_metrics_from_pg(event_ids)  
  
    # 5) Prompt bauen  
    prompt = build_prompt(query, graph_ctx, metrics)  
  
    # 6) LLM-Antwort  
    answer = call_llm(prompt)  
  
    return answer
```

7.3 Beispiel für einen Prompt-Aufbau

- Abschnitt 1: Nutzerfrage
- Abschnitt 2: Aktuelle Marktsituation (aus Kursdatenbank, optional)
- Abschnitt 3: Historische Episoden (aus Neo4j/Postgres)
- Abschnitt 4: Instruktion an das LLM

Beispiel (stark vereinfacht):

Nutzerfrage: {{query}}

Historische Episoden (Top-Treffer): 1. {{event_1.event_name}} - Regime: {{event_1.market_regime}}, Phase: {{event_1.market_phase}}, Return: {{event_1.metrics.event_return_pct}}. 2. ...

Aufgabe: - Analysiere die Gemeinsamkeiten dieser Episoden. - Erläutere die typischen Mechanismen und Risiko-/Chancenprofile. - Leite daraus allgemeine Muster ab, ohne konkrete zukünftige Kursziele zu nennen.

8. Betrieb, Monitoring und Fehlerszenarien

8.1 Typische Fehlerquellen

- JSON nicht valide → Datei nach `/error/`, Log + Benachrichtigung.
- Enum-Verstöße (z. B. `risk_profile`) → in n8n korrigieren oder hart blocken.
- fehlende base-Datei für eine refined-Datei → Warnung, ggf. manueller Check.
- Neo4j/Qdrant nicht erreichbar → Retry/Fehlerpfad in n8n.

8.2 Logging

Empfohlen:

- n8n-Log „per Workflow-Run“ aktiv halten.
- In Postgres `raw_files` nutzen, um alle verarbeiteten Dateien zu sehen.
- Für den Graph-RAG-Service: Request/Response-Logs (anonymisiert) zur Fehleranalyse.

8.3 Schrittweise Inbetriebnahme

1. Tabellen in PostgreSQL anlegen (`ohlcv_1h`, `crypto_market_events`, `event_metrics`, `raw_files`).
2. n8n-Workflow ohne Neo4j/Qdrant-Teil aufbauen und mit Test-JSONs verifizieren.
3. Neo4j-Upserts ergänzen und Graph mit wenigen Events testen.
4. Qdrant-Embeddings ergänzen und Vektorsuche testen.
5. Graph-RAG-Service als letzten Schritt aufsetzen und iterativ verfeinern.

Damit haben Sie einen durchgängigen, technisch präzise beschriebenen Pfad von den zweimonatlich erzeugten JSON-Files im Dropfolder bis hin zu einem produktiv nutzbaren Graph-RAG-System für BTC/ETH/SOL.